

Holt 2.0

Ignacio Díaz-Emparanza and Allin Cottrell

2026-09-14

1 Introduction

This package implements the forecasting method devised by C. C. Holt (1957), namely double exponential smoothing with level and trend and components. It requires time-series data. If you want triple exponential smoothing (with a seasonal component), as in Winters 1960, try the Winters package.

A prior version of this package (1.0) was written by Ignacio Díaz-Emparanza in 2015. This revision was written by Allin Cottrell, exploiting modern `hansl` syntax and adding functionality for automatic determination of the Holt smoothing parameters. See Section 7 below for help with updating scripts for use with version 2.

The filtering recursions and forecasting equation for Holt's method are shown below. The notation is based on (Hyndman and Athanasopoulos, 2021, section 8.2): ℓ denotes the level and b the trend or slope; α and β are the smoothing parameters.

$$\begin{aligned}\ell_t &= \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \\ b_t &= \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1} \\ \hat{y}_{t+h|t} &= \ell_t + hb_t\end{aligned}$$

Like most functions in `gretl`, `Holt` confines its operations to the sample range in force when the function is called. We'll refer to the (1-based) start and end of this range as t_1 and t_2 , respectively.¹ In addition, let T denote the date of the last valid observation of the variable to be predicted, y .

There are two cases to be considered:

1. $T < t_2$: in this case there's a 'tail' of observations for which values of y_t are not available. Winters will by default produce predictions for these observations.
2. $T = t_2$: there's no tail for genuine out-of-sample prediction, but you can specify a number of observations, `ntest`, to reserve for pseudo-out-of-sample prediction. In that case the Holt training recursion will be terminated at observation $T - \text{ntest}$.

If you want case 1 but your dataset doesn't contain any extra trailing observations it's straightforward to add them using the `dataset addobs` command, or via the `Add observations` item under the `Data` menu in the graphical interface.

Conversely, if the dataset does contain extra trailing observations but you want case 2, specifying a positive value for `ntest` will result in the extra observations being ignored and pseudo-out-of-sample forecasts being produced.

2 The public functions

The primary function has the following signature:

```
function bundle Holt (const series y, const bundle specs[null])
```

¹This range can be set via the `smpl` command. If `smpl` has not been invoked since a dataset was opened t_1 will equal 1 and t_2 will equal the total number of observations.

The required argument `y` specifies the series to be processed. The second argument—which can be omitted, as indicated by the `null` modifier—may contain one or more of the following options:

- `theta` (matrix): if supplied, this must be a 2-vector holding the smoothing parameters $\theta = (\alpha, \beta)$, with each parameter in $[0, 1]$. If `parms` is not given these parameters are determined automatically.
- `ntest` (integer): specifies the number of observations at the end of the range of `y` values to reserve for testing via pseudo-out-of-sample forecasting. The default value is 0.
- `verbose` (integer): controls the verbosity of output, with levels 0, 1 (the default), 2 or 3.

The return value from `Holt` is a bundle containing the following elements:

- `theta`: a 2-vector holding the smoothing parameter values.
- `init`: a 2-vector of holding the initial level and trend.
- `ntest`: a non-negative integer indicating how many observations were reserved for pseudo-out-of-sample forecasting.
- `H`: a $T \times 4$ matrix. Column 1 holds the data passed as `y`; columns 2 and 3 hold the estimated level and trend; and column 4 holds forecasts. More details on this matrix are provided below.
- `RMSE_train`: the root mean squared error for the 1-step ahead forecasts made using the training data.
- `MAE_train`: the mean absolute error for the 1-step ahead forecasts made using the training data.
- `yname`: the name of the input series.

If `ntest` is non-zero, two additional items are included in the bundle, namely the RMSE and MAE for the pseudo-out-of-sample forecasts, under the keys `RMSE_test` and `MAE_test`.

A simple plotting function is also supplied:

```
function void Holt_plot (bundle *b, int ptype)
```

The first argument should be a pointer to a bundle obtained from `Holt` and the second argument should be either 1 for a plot of actual and forecast values, or 2 for a tripartite plot of the level, trend and irregular components.

3 Command-line usage

We'll illustrate with replication of the Australian population example given in [Hyndman and Athanasopoulos \(2021, section 8.2\)](#). Here's the `hansl` code:

```
include Holt.gfn
open auspop.gdt --frompkg=Holt
dataset addobs 10
bundle hb = Holt(pop)
Holt_plot(&hb, 1)
Holt_plot(&hb, 2)
```

The `auspop` dataset contains 58 annual observations from 1960 to 2017, which is used for training the model. The ten added observations, 2018 to 2027, are then used for out-of-sample forecasting. Since `theta` is not specified, both the parameters and the initializer for the Holt recursion will be optimized automatically; see section 6 for details.

One might choose, instead, to use the `ntest` option to get `Holt` to generate (say) four pseudo-out-of-sample forecasts:

```
open auspop.gdt --frompkg=Holt
bundle hb = Holt(pop, _(ntest=4))
```

This generates forecasts for 2014 to 2017, and the RMSE and MAE for these testing forecasts are printed.

4 Output details

Table 1 shows what is printed by `Holt` as a function of the `verbose` option. An example of the table of results displayed when `verbose` is 1 or greater is presented in the lower part of listing 1. The matrix `H` that's included in the bundle returned by `Holt` is basically a matrix version of this table. If you want to grab the columns of `H` as a list of series you can do something like

```
bundle hb = Holt(pop)
list L = mat2list(hb.H)
list L print # show all the series in L
```

Table 1: Effect of the `verbose` option

<i>verbose</i>	<i>cumulative effect</i>
0	Nothing is printed.
1	Parameters and initializer are reported, also a table of results (see below) and the SSE and RMSE for the training data.
2	If optimization is performed, partial information on progress of the iterative procedure is shown.
3	If optimization is performed, progress of the optimizer is shown in full.

5 GUI usage

When you install this package you should get the option of attaching it as the menu item `Variable/Filter` in `gretl`'s main window. Clicking this item brings up the dialog box shown in Figure 1.

Hopefully the choices available in the dialog are fairly self-explanatory, but clicking the `Help` button will bring up a brief explanation of each control. On clicking `OK` you should get a window showing the results (Figure 2). The toolbar at the top of this window includes a `Graph` button which gives access to two types of plot, one showing actual and forecast y and one showing the level, trend and irregular components.

6 Technical details

This section offers some details on the rule-of-thumb initialization with which the Holt recursion is started, and the optimization that is carried out if `theta` is not specified by the caller.

Listing 1: Partial listing of results from the first Australian population example in Section 3.

```
Holt double exponential smoothing
Training range: 1960 to 2017
Out-of-sample forecast: 2018 to 2027

automatic parameter values:
   $\alpha = 1$ ,  $\beta = 0.325466$ 

estimated initializer:
  level = 10.0527, trend = 0.223798

training RMSE = 0.0620315, MAE = 0.0432489
```

	y	level	trend	forecast
1960	10.27648	10.27648	0.2237983	10.27648
1961	10.48300	10.48300	0.2181758	10.50028
1962	10.74200	10.74200	0.2314627	10.70118
1963	10.95000	10.95000	0.2238264	10.97346
1964	11.16700	11.16700	0.2216046	11.17383
1965	11.38800	11.38800	0.2214078	11.38860
1966	11.65100	11.65100	0.2349447	11.60941
1967	11.79900	11.79900	0.2066471	11.88594
1968	12.00900	12.00900	0.2077384	12.00565
1969	12.26300	12.26300	0.2227950	12.21674
1970	12.50700	12.50700	0.2296965	12.48579
...				
2006	20.69790	20.69790	0.2642719	20.64034
2007	20.82760	20.82760	0.2204733	20.96217
2008	21.24920	21.24920	0.2859332	21.04807
2009	21.69170	21.69170	0.3368904	21.53513
2010	22.03175	22.03175	0.3379188	22.02859
2011	22.34002	22.34002	0.3282704	22.36967
2012	22.74247	22.74247	0.3524137	22.66829
2013	23.14590	23.14590	0.3690165	23.09489
2014	23.50414	23.50414	0.3655081	23.51492
2015	23.85078	23.85078	0.3593691	23.86965
2016	24.21081	24.21081	0.3595826	24.21015
2017	24.59893	24.59893	0.3688719	24.57039
2018				24.96780
2019				25.33668
2020				25.70555
2021				26.07442
2022				26.44329
2023				26.81216
2024				27.18104
2025				27.54991
2026				27.91878
2027				28.28765

Holt

Select arguments:

y (series) +

parms ▾

level, α - +

trend, β - +

ntest, see Help - +

verbose - +

Help Apply Close OK

Figure 1: Holt dialog, under the Variable/Filter menu

Holt double exponential smoothing
 Training range: 1960 to 2017
 Number of forecasts: 0

automatic parameter values:
 $\alpha = 1$, $\beta = 0.325466$

estimated initializer:
 level = 10.0527, trend = 0.223798

training RMSE = 0.0620315, MAE = 0.0432489

	y	level	trend	forecast
1960	10.27648	10.27648	0.2237983	10.27648
1961	10.48300	10.48300	0.2181758	10.50028
1962	10.74200	10.74200	0.2314627	10.70118
1963	10.95000	10.95000	0.2238264	10.97346
1964	11.16700	11.16700	0.2216046	11.17383
1965	11.38800	11.38800	0.2214078	11.38860
1966	11.65100	11.65100	0.2349447	11.60941
1967	11.79900	11.79900	0.2066471	11.88594
1968	12.00900	12.00900	0.2077384	12.00565

Figure 2: Holt output window

First, the rule-of-thumb initialization is as follows, where τ denotes the first period in which y is observed.

- The initial level, $\ell_{\tau-1}$, is set to the first observed value y_{τ} .
- The initial trend, $b_{\tau-1}$, is set to the first observed difference, $y_{\tau+1} - y_{\tau}$.

As regards optimization, two cases arise.

- If `theta` is not specified, the parameter values and initializer are optimized jointly, with the criterion of minimizing the sum of squared forecast errors for the training range. The starting point for the parameters is $\alpha = \beta = 0.2$; the starting point for the initializer is as described above. Optimization is performed using the L-BFGS algorithm (see the chapter on Numerical methods in the *Gretl User's Guide* for details), with α and β constrained to lie in $[0, 1]$.
- If `theta` is given, the initializer is just the rule-of-thumb value; no optimization is performed.

7 Updating from Holt 1.0

If the caller specifies the smoothing parameters, θ , and if the `init` argument in version 1.0 is set to 0 (meaning that the initializer is set via the rule of thumb described in the previous section), the two versions of the package will produce the same sequence of one-step-ahead forecasts. The relevant `hansl` code is shown under the heading **Case 1** in Listing 2. Note that while in version 1.0 the `Holt` arguments are strictly positional, in version 2.0 they are in effect named, since they appear as elements of a `gretl` bundle (apart from the first required series argument). This example also shows how to extract the forecast series from the respective returned bundles.

Case 2 in the listing shows how to generate testing forecasts using each version of the package. In this case we don't specify θ , but accept the default behavior of the `Holt` function, which differs by version. In version 1.0 θ defaults to the fixed vector $(0.3, 0.1)$ and the initializer is set via an OLS regression using the first half of the training observations. In version 2.0 θ and the initializer are jointly optimized using all of the training data.

Case 3 exhibits the generation of an out-of-sample forecast after adding empty "observations" to the dataset. For both versions, only the input series needs to be specified. The default behavior differs as described for **Case 2**.

The content of the returned bundle also differs across the versions of the package. In version 1.0 two series are included: the original series and the Holt forecast, while version 2.0 gives a four-column matrix holding the original series, level, trend and forecast as in Hyndman and Athanasopoulos (2021). Where version 1.0 reports the mean square error (MSE), version 2.0 reports the RMSE.

Forecast stderr:

$$\sigma_h^2 = \sigma^2 \left[1 + (h-1) \left\{ \alpha^2 + \alpha\beta h + \frac{1}{6}\beta^2 h(2h-1) \right\} \right]$$

8 Change log

- v2.0, 2026-09-26: Initial release of substantially modified version.

References

Holt, C. C. (1957) 'Forecasting seasonals and trends by exponentially weighted moving averages'. ONR Memorandum, Vol. 52, Carnegie Institute of Technology. URL <https://doi.org/10.1016/j.ijforecast.2003.09.015>.

Listing 2: Holt 1.0 examples with corresponding 2.0 code

Common initial code

```
include Holt.gfn
open data3-6.gdt
```

Case 1, producing the same forecast sequence:

```
# version 1.0
bundle bh1 = Holt(Ct, 0.7, 0.1, 0)
series fcast1 = bh1.Ct_Holt
# version 2.0
bundle bh2 = Holt(Ct, _(theta = {0.7, 0.1}))
series fcast2 = bh2.H[,4] # or "= bh2.Ct_fcast" in gretl 2026c
```

Case 2, automatic parameters and 4 testing forecasts:

```
# version 1.0
bundle bh1 = Holt(Ct, , , , 4)
# version 2.0
bundle bh2 = Holt(Ct, _(ntest=4))
```

Case 3, automatic parameters and 4 out-of-sample forecasts:

```
dataset addobs 4
# version 1.0
bundle bh1 = Holt(Ct)
# version 2.0
bundle bh2 = Holt(Ct)
```

Hyndman, R. J. and G. Athanasopoulos (2021) 'Forecasting: principles and practice, 3e'. URL <https://otexts.com/fpp3/>. Accessed: 2026-09-14.

Winters, P. R. (1960) 'Forecasting sales by exponentially weighted moving averages', *Management Science* 6(3): 324–342. URL <https://doi.org/10.1287/mnsc.6.3.324>.